



Buffer的创建

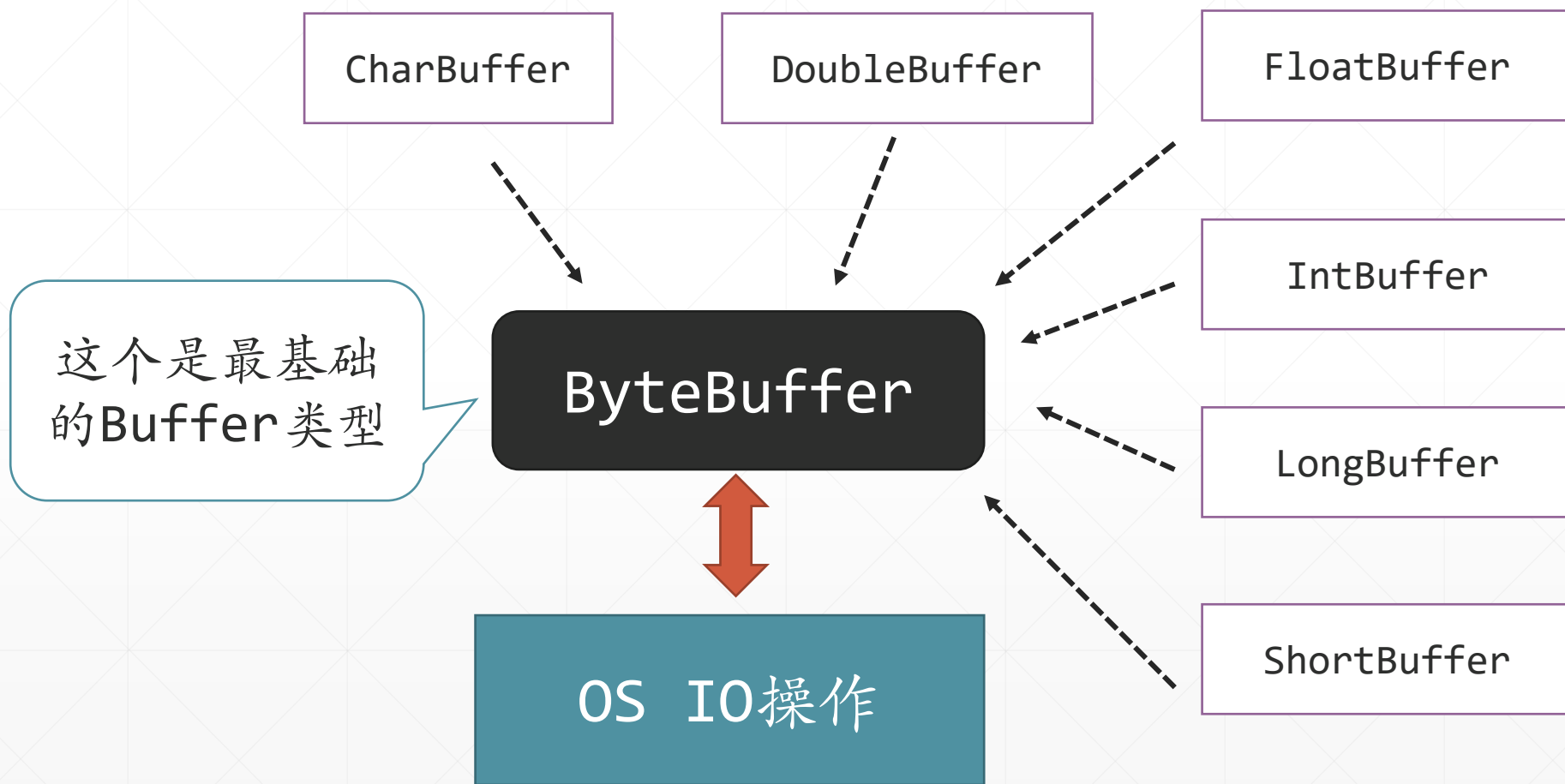
北京理工大学计算机学院
金旭亮

什么是Buffer?

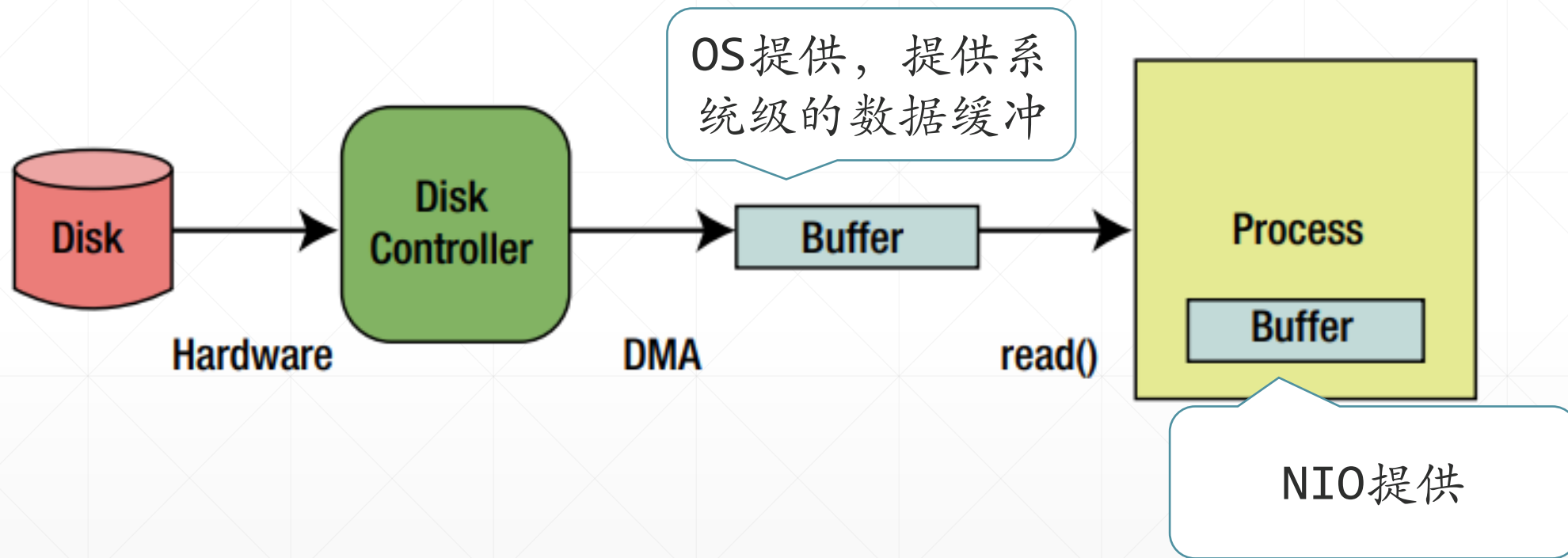


一个Buffer对象是固定尺寸的数据的容器

JDK中的Buffer类型



两种不同地位与作用的Buffer



JDK中的Buffer，用于在Java程序进程内部缓存数据。

JDK中的Buffer类



Buffer类能存储的数据类型是多样的。



Buffer类是抽象类，并不能直接实例化，而其子类：
ByteBuffer、CharBuffer、DoubleBuffer、FloatBuffer、
IntBuffer、LongBuffer 和 ShortBuffer 也是抽象类。
真正实例化的，其实是HeapByteBuffer这样的子类。

实例化Buffer的方法



使用wrap()方法

基于现有集合（或数组）创建一个Buffer，但是不分配任何新的空间来储存数据元素。

使用allocate()方法

创建一个Buffer对象，并分配一个私有的空间，其大小由其参数所指定。

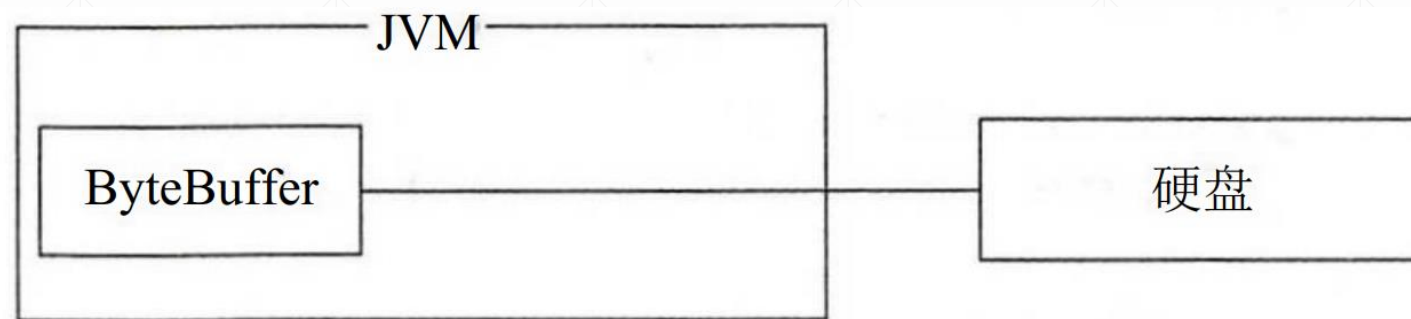


Buffer创建之后，调用put方法写，调用get方法读。已经读完的数据，如果需要再读一遍，可以调用rewind()方法。

直接缓冲区与非直接缓冲区



非直接缓冲区（默认值）



直接缓冲区

创建直接与间接缓冲区的示例

//测试isDirect和hasArray方法

```
private static void testDirectAndHasArray() {  
    byte[] arr = {0, 1, 2, 3, 4, 5};  
  
    ByteBuffer buffer1 = ByteBuffer.wrap(arr);  
    System.out.println(buffer1.isDirect()); //false  
    System.out.println(buffer1.hasArray()); //true  
  
    ByteBuffer buffer2 = ByteBuffer.allocate(6);  
    System.out.println(buffer2.isDirect()); //false  
    System.out.println(buffer2.hasArray()); //true  
  
    ByteBuffer buffer3 = ByteBuffer.allocateDirect(6);  
    System.out.println(buffer3.isDirect()); //true  
    System.out.println(buffer3.hasArray()); //false  
}
```

通过allocate()或者wrap()函数创建的缓冲区通常都是间接的，间接的缓冲区使用内部数组存储数据。

如果需要的话，也可以创建直接缓冲区，它将数据直接写入到操作系统的缓冲区中。

Buffer与底层数组之间的关系-1

```
ByteBuffer buffer1 = ByteBuffer.allocate(10);
if (buffer1.hasArray()) {
    //buffer1 array: [B@6d311334
    System.out.println("buffer1 array: " + buffer1.array());
    //Buffer1 array offset: 0
    System.out.println("Buffer1 array offset: " +
        buffer1.arrayOffset());
    MyBufferUtils.printBufferInfo(buffer1);

    System.out.println();
}
```



```
buffer1 array: [B@6d311334
Buffer1 array offset: 0
=====
Capacity: 10
Limit: 10
Position: 0
Remaining: 10
=====
```

BufferAndItsArray.java

Buffer与底层数组之间的关系-2

```
byte[] bytes = new byte[200];
ByteBuffer buffer2 = ByteBuffer.wrap(bytes, offset: 10, length: 50);
if (buffer2.hasArray()) {
    //buffer2 array: [B@4dd8dc3
    System.out.println("buffer2 array: " + buffer2.array());
    //Buffer2 array offset: 0
    System.out.println("Buffer2 array offset: " +
        buffer2.arrayOffset());
    MyBufferUtils.printBufferInfo(buffer2);
}
```



```
buffer2 array: [B@6d03e736
Buffer2 array offset: 0
=====
Capacity: 200
Limit: 60
Position: 10
Remaining: 50
=====
```

BufferAndItsArray.java

注意：

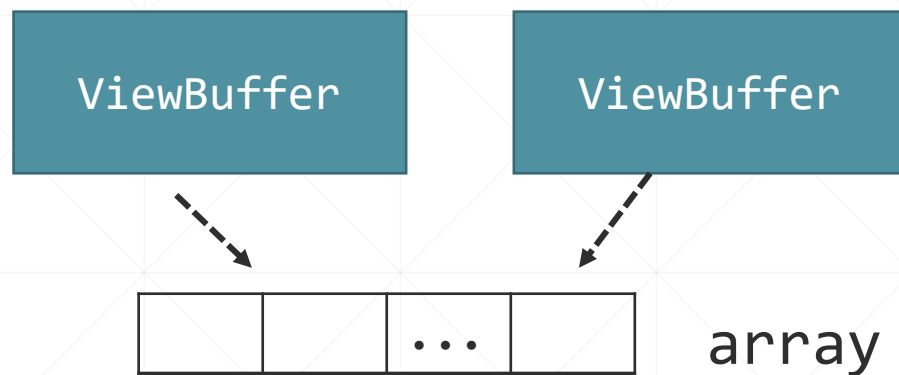


缓冲区并不是多线程安全的。如果您想以多线程同时存取特定的缓冲区，您需要在存取缓冲区之前进行同步（例如给缓冲区对象加锁）。



所有的缓冲区都是可读的，但并非所有都可写。每个具体的缓冲区类都通过执行`isReadOnly()`来标示其是否允许该缓存区的内容被修改。

创建Buffer视图



```
buffer.asXXXBuffer()
```